

Judge the Votes: A System to Classify Bug Reports and Give Suggestions

Emre Dinç

Department of Computer Engineering
Bilkent University
Ankara, Türkiye
emre.dinc@ug.bilkent.edu.tr

Eray Tüzün

Department of Computer Engineering
Bilkent University
Ankara, Türkiye
eraytuzun@cs.bilkent.edu.tr

Abstract—Context: A large number of bug reports are submitted in software projects on a daily basis, examined manually by practitioners to determine if code changes are needed (valid) or not (invalid). Invalid bug reports cause a great waste of resources, thus automated bug report classification is widely studied. However, there exists a gap using LLMs and prompting techniques for bug report classification.

Objective: The aim of this study is to apply various LLM and ML methods to classify bug reports as valid or invalid, and to determine strengths and weaknesses of the LLMs for bug report classification.

Method: We trained machine learning models on the dataset, and established a retrieval augmented generation pipeline by indexing the training data to see how it affects LLM performance in classification. Lastly, we compared the performance of the majority vote of the three models we chose and compared it with the Judge LLM which is provided both with examples and models votes.

Results: Fine-tuned transformers proved to be the most effective classifiers, with RoBERTa achieving the highest F_1 score of 0.909. In contrast, zero-shot LLMs have shown a poor performance, with F_1 scores ranging from 0.531 to 0.737. Although RAG-based few-shot prompting significantly boosted LLM performance, lifting the top score to 0.815, they still did not surpass the fine-tuned models. Finally, our Judge LLM pipeline yielded an F_1 score of 0.871.

Conclusions: Our evaluation confirms that fine-tuned semantic classifiers remain the most cost-effective and highest-accuracy solution for binary bug-report classification. Using RAG with LLMs substantially improves model performance, although the scores do not surpass our top fine-tuned models.

Index Terms—Bug Report Classification, Explainable AI, Bug Validity, Retrieval Augmented Generation, Reasoning with LLMs, Invalid Bugs

I. INTRODUCTION

Bug reports are submitted by users or software practitioners when a software product does not behave as expected. Large projects receive tens or even hundreds of thousands of reports per year. For example, Mozilla Firefox logged 224,408 new bug reports in a 24 month period between January 2013 and December 2014 [1]. Since each report traditionally requires manual inspection, triaging consumes substantial time and effort [1], [2]. Consequently, teams waste considerable resources investigating bug reports that do not require code changes [3].

Over the past two decades, numerous studies have applied machine learning (ML) to automate report classification [1],

[3], [4], using resolved reports from open-source projects to learn discriminative text features [5], [6]. More recently, large language models (LLMs) have been used for this task [7], [8], but differences in data splits and model configurations hinder direct comparisons. Laiq et al. [9] addressed this by training ML models and fine-tuning RoBERTa on the same dataset, with RoBERTa slightly surpassing the ML models [9].

One emerging frontier is Retrieval-Augmented Generation (RAG) with LLMs [10], [11]. Yet to our knowledge, no prior work applies this to bug triage. We harness a Judge LLM by embedding our entire training corpus (8,000 Firefox reports) with a Sentence-Transformer [12] and indexing it with FAISS [13], then retrieving the top-5 (to strike a balance between context window and performance) semantically similar bug reports with their labels as in-context examples. All embeddings are L2-normalized and queried with IndexFlatIP [13], so neighbor selection is by cosine similarity. We test four LLMs to see how they perform with only a base prompt vs. with in-context examples for each of the 2,000 bug reports in the test set. We then combine these context examples with the discrete votes of three classifiers (Random Forest, SVM, and RoBERTa) to prompt the best-performing LLM for a final binary classification decision.

Most existing pipelines end at classification, leaving bug report submitters without guidance until a human reviewer intervenes. To close this gap, our Judge LLM emits not only a binary decision but also a concise, user-focused explanation of its reasoning and actionable suggestions for improving the bug report. Thus, an LLM, as we discuss in Section V, can be used not as the final decision-maker but as an explainer. Taken together, these components position our study to examine how reasoning, retrieval, and ensemble strategies jointly influence the performance and interpretability of automated bug triage, leading to the following research questions.

RQ1: How do reasoning LLMs perform compared to prior ML and LLM classifiers with few-shot prompting?

RQ2: What is the effect of integrating a Judge LLM into a majority-voting ensemble versus using majority-voting alone?

RQ3: How does few-shot prompting with RAG impact LLM performance relative to zero-shot prompting?

We make the following contributions.

1) Demonstrating that a reasoning LLM can effectively

decide on bug validity when supported with few-shot prompting.

- 2) Relying solely on textual features to streamline training and maintain model explainability without sacrificing accuracy.
- 3) Utilizing LLM reasoning to generate concise and user-focused explanations of every decision with actionable suggestions.
- 4) Studying the instances where the Judge LLM and majority voting disagree, aiming to identify the bug report characteristics that lead to classification failures.

II. RELATED WORK

A. ML Models in Bug Report Classification into Valid and Invalid Categories

Bug reports have been studied in different contexts. Sun et al. [14] and Kucuk et al. [15] studied duplicate bug reports, while Zanetti et al. [4] studied the assignment of bug reports using collaboration networks. Antoniol et al. [3] focused on the task of classifying bug reports into bug and non-bug categories, valid and invalid bugs in our terminology. They extracted textual features and identified critical words that help ML models classify bug reports with precision ranging between 64-98% and recall between 33-97% for a sample of 1,800 bug reports.

Fan et al. [1] focused on using different ML models to classify bug reports. They took the work of Zanetti et al. [4] as a base and extended it to include more features for determining bug report validity. Their study compared the performance of their random forest (RF) model against previous models and achieved a 15% improvement. Additionally, they identified the key features that determine bug report validity, finding textual parts of the bug report (description and summary) and submitter experience as the most important among 33 features.

Laiq et al. [9] compiled existing studies on bug report classification and implemented various ML models, including SVM (Support Vector Machines), NB (Naive Bayes), RF (Random Forest), AutoML, and RoBERTa, evaluating them on a unified scale. Their results showed RoBERTa achieves the highest performance in terms of accuracy. They also assessed ML models based on additional factors such as explainability, scalability, generalizability, and cost [9].

Otoom et al. [5] and Pandey et al. [6] leveraged SVM and RF to classify bug reports as bug-related or non-bug-related. Large language models have also been applied to the classification of bug reports. Laiq et al. [9] fine-tuned RoBERTa, Meng et al. [7] fine-tuned BERT embeddings. Lastly, Mosel et al. [16] fine-tuned a 128 context-sized BERT for the bug report classification task.

B. Information Retrieval Techniques and Their Use in Bug Report Classification

Traditional information-retrieval techniques have long underpinned defect-centric analytics. Duplicate-report retrieval [14], IR-based issue triage [6], and TF-IDF or word-embedding baselines for bug assignment [4] all treat a bug

report as a query over a corpus of prior issues. These studies demonstrate that even shallow lexical similarity can boost triage automation, but they leave the generation of explanations or suggestions entirely to human reviewers.

Beyond classical retrieval, the natural language processing (NLP) community has converged on RAG as a scalable strategy for knowledge intensive tasks. RAG injects top- k passages retrieved by dense or sparse search into the prompt of a large language model, enabling factual grounding without exhaustive fine-tuning [10]. Guu et al. [11] propose a system, REALM, which further shows that language model pre-training can itself be retrieval-aware, jointly optimizing a bi-encoder and generator to read from an external memory of documents [11].

Within software engineering, early attempts to use RAG have centered on code search and security auditing. Wang et al. [17] combine contrastive retrieval with GPT-3 to answer API-usage questions, while Arzanipour et al. [18] integrate vulnerability snippets into GPT-4 prompts for secure-coding advice. These works confirm that retrieval can inject domain-specific context, yet their evaluation stops at code-level tasks and does not address unstructured bug descriptions.

Despite these advances, the literature still lacks a retrieval augmented pipeline that *jointly* tackles binary bug-validity classification and generates natural-language feedback for reporters. Prior studies on invalid-bug detection rely either on classical learners with handcrafted features [1], [9] or on fine-tuned transformers without external context [7]. Conversely, existing RAG systems in software engineering focus on code understanding or vulnerability analysis and do not incorporate a reasoning LLM that weighs auxiliary model votes while producing human-readable rationales [17], [18]. Our work therefore bridges two open gaps: it is, to the best of our knowledge, the first to couple a RAG pipeline with a chain-of-thought LLM for bug report validity decisions, and the first to extend the bug classification output beyond a label by providing natural language explanations.

III. METHODOLOGY

In this section, we outline our methodology for classifying bug reports and generating an explanation for each classification. In Section III-A, we describe how we fetch the dataset via the Bugzilla API for Mozilla Firefox, corresponding to the leftmost stage of Figure 1, and detail the dataset in Section III-B. Next, we present the training and fine-tuning of ML models and LLMs in Section III-C, followed by the evaluation pipeline with different LLM configurations and the Judge LLM that aggregates votes and reasons. Finally, as shown on the right side of Figure 1, we evaluate all methods. The replication package is available at <https://doi.org/10.6084/m9.figshare.29627297.v1>.

A. Dataset Construction and Preprocessing

We retrieved 10,000 Mozilla Firefox bug reports via the Bugzilla REST API [19], filtering for statuses `RESOLVED`, `VERIFIED`, or `CLOSED` and resolutions `WORKSFORME`,

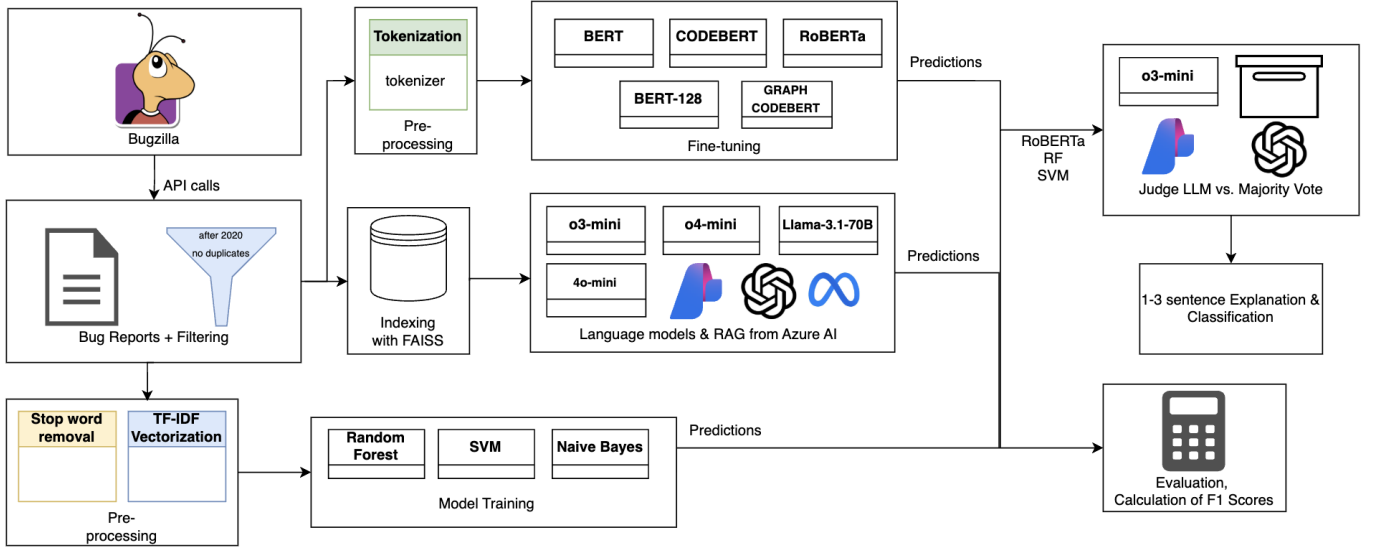


Fig. 1: The pipeline of our study

INVALID, FIXED, INCOMPLETE, and WONTFIX. For each report we retained the reporter’s id, login and real name, creation timestamp, summary, description, status, resolution, platform, and operating system. We then concatenated summary and description into a single `text` field to be used in model training. We aimed to see how the model performances change when only textual data is used, given textual data is one of the key features Fan et al. [1] found for the bug report classification.

To frame our binary validity task, we collapsed the five resolution categories into two labels following previous research [1]. FIXED and WONTFIX map to *valid* (1), and WORKFORME, INVALID, and INCOMPLETE maps to *invalid* (0). We have retrieved bug reports between dates 2020-01-01 and 2020-04-13 to provide a safe margin without bug reports being too old or too new. We impose roughly a five-year buffer between the newest report and the time of analysis to ensure that almost every issue has reached a stable final resolution, thereby reducing label noise from still-open or recently triaged tickets. A comparable multi-year cut-off was used by Zimmermann et al. [20] who excluded very recent bugs when studying reopen rates, so that only cases with settled outcomes entered their empirical model training.

Prior to modeling, we applied distinct preprocessing pipelines. For *Classical ML models (NB, RF, SVM)* we used TF-IDF vectorization with stop-word removal, converting each report’s `text` into a sparse feature vector [1], [4]. For *Transformer fine-tuning (BERT variants)* we used tokenizers for each BERT-based model, padding and truncation to 512 (or 128 for a BERT variant we fine-tuned) tokens [21].

By reducing each record to just two fields `text` and `label` we maintained a minimal feature set that enables direct comparison across classical, transformer, and LLM-based approaches.

B. Dataset Description

Each row in the dataset contains ten columns. After concatenating the summary and description into a single free-text attribute (`text`), we retained all other columns unchanged for possible future analyses; however, only the `text` field and binary `label` are consumed by the classifiers presented in later sections.

TABLE I: Dataset columns and their descriptions (with example values drawn from our filtered dataset).

Column	Description
id	Unique Bugzilla identifier.
summary	One-line headline of the issue.
description	Detailed report body and reproduction steps.
creation_time	Bug report creation timestamp (UTC).
status	Workflow state (e.g., RESOLVED, VERIFIED, CLOSED).
resolution	Final outcome (e.g., FIXED, WONTFIX, WORKFORME, INVALID, INCOMPLETE).
creator	Reporter username.
creator_detail	Reporter display name.
platform	CPU architecture the bug was produced.
op_sys	Operating system the bug was produced.

TABLE II: Binary label distribution after mapping resolutions to *valid* (FIXED/WONTFIX) and *invalid* (all others).

	Training	Test
Valid	6 841	1 363
Invalid	1 159	637

After the binary mapping (Table II) the class distribution is moderately imbalanced, about 85 % valid versus 15 % invalid in training and 68 % versus 32 % in testing due to randomization, necessitating F_1 -oriented metrics later in the paper. We did not use stratification in creating the train/test split to better simulate real-time deployments, and stratification

TABLE III: Resolution categories used by Bugzilla [4].

Resolution	Definition
FIXED	Confirmed defect corrected in code.
WONTFIX	Valid issue acknowledged but will not be addressed.
WORKSFORME	Report could not be reproduced in given environment.
INVALID	Report describes intended behavior or non-defect.
INCOMPLETE	Insufficient detail for triage or duplication search.

is shown to not change the accuracy of ML models to a considerable degree [22].

Together with the heterogeneity of Bugzilla resolutions (Table III), these statistics frame the empirical challenges addressed by our subsequent modeling and evaluation pipeline. Although the resulting label distribution is skewed toward “valid” reports, we retained the natural imbalance because resampling techniques can distort textual feature spaces and have shown only marginal benefits on bug-triage tasks when robust and threshold-independent metrics such as macro- F_1 are reported [1], [23].

The average narrative length of roughly 185 words indicates that each report offers rich contextual prose, benefiting models that exploit long-form semantics. Invalid bug reports (4,021 characters) are found to be about 3.7 times longer than the valid bug reports (1,084 characters), often due to noise like logs directly copied and pasted into the bug report.

C. ML Model Training: NB, RF, and SVM

To establish strong classical baselines, we trained three standard supervised classifiers on TF-IDF representations of the bug report text. We used **NB**, **RF**, **SVM** models which have produced good results for bug report classification tasks. Fan et al. [1] report that a SVM reaches 0.82, F_1 on Firefox data, surpassing other shallow learners by roughly four to six percentage points. On a large-scale Jira corpus, Pandey et al. [6] observe RF at 0.80, F_1 and SVM at 0.83, F_1 , confirming their robustness across projects. Otoom et al. [5] adopt Multinomial NB as a reference point, noting a 0.71 F_1 , yet emphasizing its well-calibrated probabilities, which prove valuable in downstream ensembles. Taken together, these results justify our choice: SVM provides strong discriminative power in sparse TF-IDF space, RF captures non-linear feature interactions while resisting over-fitting, and NB offers an interpretable, ultra-fast lower bound against which we can measure subsequent gains from Transformer-based models.

Each pipeline first applies a TF-IDF Vectorizer, then fits the chosen classifier. We performed 10-fold cross-validation on the training split (80% of our data) to tune hyperparameters where applicable, then retrained each model on the entire training set before evaluating on the held-out test set (20%). This classical pipeline yields fast, interpretable predictions that we later combine via majority voting.

D. Fine-Tuning Transformer-based Models

Beyond classical methods, we fine-tuned five pre-trained transformer encoders for binary bug validity classification. These include **BERT-base-uncased** [24], **RoBERTa-base** [25], **DistilBERT-base-uncased** [26], **CodeBERT**

(**microsoft/codebert-base**) [27], and a **128-token BERT variant** inspired by token-length sensitivity studies [28].

For each model, we appended a single linear classification head to the pooled output (CLS token), and fine-tuned all parameters end-to-end using the HuggingFace Trainer API [29]. We trained for 3 epochs with a learning rate of 2×10^{-5} , per-device batch sizes of 16 (train) and 32 (eval) with cross-entropy loss function, and weight decay of 0.01. Text inputs were truncated or padded to a maximum length of 512 tokens (except the 128-token variant). We evaluated each model on the test set, computing accuracy, precision, recall, and several F_1 variants, to compare directly with both the classical classifiers and the Judge LLM.

E. LLM-Based Classification with Different Prompting Techniques

We chose to use LLMs in bug report classification due to their prevalence and promising ability in reasoning about decision-making tasks [30]. We have experimented with both zero-shot and few-shot prompting techniques to see how LLM performance varies. *Zero-shot prompting* is instructing the model about a specific task without giving any specific examples [31]. *Few-shot prompting* is providing some examples to the model with the prompt to help it better understand the task [32]. We fed the base prompt given below to the model in both zero-shot and few-shot settings since it includes the fundamental information from Bugzilla Docs [19] and classification heuristics.

BASE PROMPT

You are a Bugzilla triage assistant.

Task: Decide whether each incoming bug report is **VALID** or **INVALID** under Bugzilla policy *and* explain your reasoning *before* giving the classification.

Definitions (Bugzilla Docs)

VALID bug: “This bug is valid and has recently been filed.” Resolution: FIXED, WONTFIX. Criteria: reproducible, not a duplicate, genuine defect or approved enhancement.

INVALID bug: “The problem described is *not* a bug.” Resolution: WORKSFORME, INVALID, INCOMPLETE. Typical reasons: user error, as-designed behavior, cannot understand functionality

Heuristics: Mark **VALID** if the issue is reproducible, violates docs or expectations, has sufficient detail, and is not already tracked.

Mark **INVALID** if behavior is documented/intentional, lacks reproduction, or stems from third-party factors.

Output format: return exactly one JSON object on a single line:

```
{"explanation":"<1{3 sentence rationale}>"
,"classification":"VALID"|"INVALID"}
```

We have decided to use few-shot prompting supported by RAG since it is shown to increase performance for knowl-

edge intensive NLP tasks [10], [11]. We appended top-5 semantically similar bug reports from the vector index of the training corpus we created using `all-MiniLM-L6-v2` [12] as embedder with 384 dimensional vectors with FAISS vector database [13]. We used L2-normalization on all MiniLM embeddings and query the FAISS index with `IndexFlatIP` [13]. For unit-length vectors the inner product equals cosine similarity, so neighbors are selected by cosine similarity [33]. We have limited the number of bug reports retrieved to 5 most semantically similar bug reports as it is shown by [10] that retrieving five examples strikes the best balance between efficacy and accuracy. We ask for JSON formatted output to simplify parsing of the outputs. Algorithm 1 shows our setup.

Algorithm 1: User prompt construction and prediction

Input: test report r , FAISS index I , boolean few_shot
Output: JSON prediction p

```

1 prompt  $\leftarrow$  BASE_PROMPT;
2 if  $few\_shot$  then
3    $\mathbf{v} \leftarrow \text{EMBED}(r)$ ;
4    $N \leftarrow I.\text{TOP5COSINESIMILARITY}(\mathbf{v})$ ;
5   foreach  $n \in N$  do
6     append Report:  $n.\text{text}$  and
7       Classification:  $n.\text{label}$  to prompt;
8 append Bug Report:  $r$  and the literal
9   Classification: to prompt;
7  $\hat{p} \leftarrow \text{LLM\_CALL}(\text{prompt})$ ;
8 return  $\hat{p}$ ;
```

Since we had only Azure AI credits, we used the models available on the Azure AI Studio [34], it limited us from using some models like Gemini 2.5 pro [35] or Claude 4.0 Sonnet [36]. Nonetheless, we used four different language models with various characteristics: GPT-o3-mini, GPT-o4-mini, and GPT-4o-mini from OpenAI [37]; and Llama-3.1-70B from Meta [38]. GPT-o3-mini and GPT-o4-mini are reasoning models using chain-of-thought reasoning [39], GPT-4o-mini is a small model with 13 billion-parameters [40] and Llama-3.1-70B is trained for a wide range of natural language understanding and generation tasks [38].

F. Ensemble Voting with Judge LLM Majority-Voting

To combine the strengths of statistical learners and large-scale reasoning models, we adopt a two-stage hybrid pipeline: (i) three base classifiers SVM, RF, and fine-tuned RoBERTa issue probabilistic votes, and (ii) GPT-o3-mini acts as a judge, weighing those votes together with five training-set neighbors retrieved by the few-shot setup described in Section III-E. We have also assembled a majority vote pipeline using the three base classifiers to compare with the Judge LLM's decisions. The judge prompt given below is fed to the LLM together with the base prompt.

JUDGE PROMPT: ensemble decision instruction

Context. Three automated classifiers have analysed the same bug report; their votes are shown below. You also receive the five most similar past reports (with ground-truth labels).

Task. Decide whether the *new* report is **VALID** or **INVALID** under Bugzilla policy and *briefly justify* your choice. If you label the report **INVALID**, add one suggestion that would help the reporter turn it into a valid submission.

Output format — return exactly one JSON object on a single line:

```

{"explanation": "1{3 sentence rationale",
 "classification": "VALID"|"INVALID"}
```

a) *Rationale for the Design.*: Prior work shows that majority-voting ensembles can smooth individual model biases yet still struggle with borderline cases [6], [9]. Recent retrieval-augmented LLM studies demonstrate that injecting external context enables coherent, fact-grounded reasoning [10], [11]. Our pipeline therefore, lets GPT-o3-mini inspect both numeric evidence (the three confidence-weighted votes) and textual evidence (five semantically closest, labeled reports) before issuing a decision. We have chosen to include the votes of SVM, RF, and RoBERTa to increase diversity instead of providing the votes of similar models [41]. This design delivers competitive accuracy while producing justifications, and actionable feedback [42]. We have selected o3-mini as the judge since it shows better performance with few-shot prompting in our dataset. Even though the Llama-3.1-70B achieves better performance with zero-shot prompting, we chose o3-mini as it is shown to make better decisions as an evaluator [43].

IV. RESULTS

A. Binary Classification of Bug Reports

Table IV shows that the three classical model baselines (highlighted in yellow) cluster in a narrow band of performance: NB achieves an accuracy of 0.825, while RF and SVM reach 0.852 and 0.829, respectively. Within these three machine learning models; RF records the highest binary F_1 (0.894) and the best precision (0.874), whereas NB exhibits the strongest recall (0.948).

The transformer encoders are the next model family (highlighted in blue). Standard BERT attains 0.899 binary F_1 ; its 128-token variant matches that score to three decimal places, while CodeBERT and GraphCodeBERT post 0.902 on the same metric. RoBERTa outperforms all other systems, topping every aggregate column with 0.870 accuracy and 0.909 binary F_1 . In contrast, the zero-shot LLM deployments (highlighted in green) have the lowest scores, with GPT-4o-mini (0.531) marking the lowest binary F_1 . Few-shot retrieval narrows this gap, GPT-o3-mini rises from 0.614 to 0.815, yet none of the LLM runs surpass the fine-tuned transformers reported above.

Figure 2 shows how the prediction of an LLM (GPT-o3-mini) changed when it is provided with top-5 bug reports vs. zero-shot prompting. With zero-shot prompting the model

TABLE IV: Performance of 16 models on the test set. Best value in each metric column is **bold**.

Model	Accuracy	F ₁ (bin.)	F ₁ (micro)	F ₁ (macro)	Precision	Recall
Naive Bayes	0.825	0.881	0.825	0.776	0.825	0.944
Random Forest	0.852	0.894	0.852	0.824	0.874	0.915
SVM	0.829	0.878	0.829	0.796	0.856	0.901
BERT	0.858	0.899	0.858	0.828	0.871	0.930
RoBERTa	0.870	0.909	0.870	0.841	0.873	0.948
CodeBERT	0.860	0.902	0.860	0.828	0.865	0.943
GraphCodeBERT	0.861	0.902	0.861	0.831	0.870	0.936
BERT-128	0.858	0.899	0.858	0.830	0.875	0.925
GPT o3-mini (few-shot)	0.741	0.815	0.741	0.690	0.792	0.840
GPT o3-mini (zero-shot)	0.469	0.614	0.469	0.382	0.609	0.619
GPT o4-mini (few-shot)	0.673	0.759	0.673	0.625	0.762	0.756
GPT o4-mini (zero-shot)	0.467	0.599	0.467	0.401	0.614	0.585
Llama-70B (few-shot)	0.712	0.797	0.712	0.650	0.766	0.831
Llama-70B (zero-shot)	0.604	0.737	0.604	0.464	0.672	0.817
GPT-4o-mini (few-shot)	0.614	0.729	0.614	0.528	0.698	0.763
GPT-4o-mini (zero-shot)	0.397	0.531	0.397	0.342	0.564	0.502

tends to classify the bug report as valid. Nevertheless, it is able to provide correct reasoning for INVALID classification.

Original bug report

worker stuck cloning for several tasks, then succeeds for later tasks
Machine i-002d25df661d9665b failed its first 5 tasks like [this] [LINK 1]:
[vcs 2020-01-22T20:39:38.617Z] ensuring [LINK 2] is available
at /builds/worker/workspace/build/src [vcs 2020-01-22T20:39:39.207Z]
(cloning from upstream repo [LINK 3]) [vcs 2020-01-22T20:39:39.730Z]
(sharing from new pooled repository [LINK 4]) [vcs 2020-01-22T20:39:39.826Z]
applying clone bundle from [LINK 5] [vcs 2020-01-22T20:39:46.722Z] 555232 files to transfer, 3.18 GB of data
[taskcluster:error] Task timeout after 7,200 seconds. Force killing container.
or:
[vcs 2020-01-23T00:56:12.812Z] ensuring [LINK 6] is available
at /builds/worker/workspace/build/src [vcs 2020-01-23T00:56:12.931Z]
(cloning from upstream repo [LINK 7])
[taskcluster:error] Task timeout after 3,600 seconds. Force killing container.
Use [LINK 8] to get a list of all tasks.
The last 2 tasks succeeded and no other machines showed at least 3 failures
during that time except i-0b3dbdcff3c307670.
Was that an issue with our config or with AWS itself?

o3-mini zero-shot

Ground-truth: INVALID

Prediction: **VALID**

Model explanation: The report includes detailed logs and a clear scenario showing reproducible behavior with multiple tasks failing before succeeding, along with a specific question about the configuration, which indicates that the problem is neither documented nor expected behavior. Therefore, it qualifies as a valid bug report.

o3-mini few-shot (with RAG)

Ground-truth: INVALID

Prediction: **INVALID**

Model explanation: The report describes intermittent and environment-related issues (timeouts and delayed cloning), and it asks whether the issue is with our config or AWS rather than pointing to a clear reproducible bug in the code. Since it does not specify a definite software bug but rather discusses operational conditions, it is classified as INVALID.

Fig. 2: Effect of RAG on the same Firefox report.

B. Judge LLM vs. Majority Voting

In Table V, we report the performance of the majority vote and the Judge LLM on the 2,000-report test set. Majority

voting pushes accuracy to 0.870 and binary F₁ to 0.906, being just short of the top-scoring RoBERTa. GPT-o3-mini in plain few-shot mode falls short of these marks, yet the Judge variant, which consumes the model votes plus the retrieval block, closes most of the gap and attains 0.816 accuracy and 0.871 binary F₁.

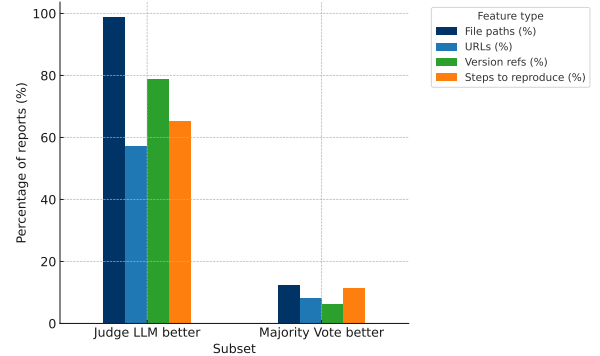


Fig. 3: Distinct content characteristics of disagreement subsets.

We performed further analysis to see that there are 258 disagreements between the Judge LLM and majority vote. Majority vote wins 70.9% of these disagreements. The judge LLM seems to classify the long bug reports as invalid even more than the original distribution for length in the dataset, an example for this is the Judge LLM’s explanation that the bug report is a log and not a valid bug report with steps to reproduce and other required fields when the bug report was actually valid. The Judge LLM is also seen to favor the bug reports including technical details such as file paths. Figure 3 shows bug reports including technical details are classified as valid more by a large ratio compared to the majority voting. Lastly, we see that the Judge LLM tends to err on the side of validity, as it has 257 false positives and 112 false negatives; while the majority voting has 72% false negative rate and tends to misclassify valid bug reports as invalid.

TABLE V: Ensemble results on the test set

Model / Setting	Accuracy	F_1 (bin.)	F_1 (micro)	F_1 (macro)	Precision	Recall
Majority Vote (SVM + RF + RoBERTa)	0.870	0.906	0.870	0.846	0.868	0.925
GPT-o3-mini Judge w/ votes	0.816	0.871	0.816	0.772	0.836	0.907

V. DISCUSSION

A. Comparison with Previous Work

We have evaluated all high-performing models from previous research on the same dataset to compare them more clearly. Our results have been consistent with the latest research by Laiq et al. [9], showing RoBERTa is the best model for bug classification task. Even though their scores are not as high as transformer based models, SVM and RF achieved solid performance, verifying the previous research by Fan et al. [1]. We improved Fan et al.’s research [1] by showing that high performance can be achieved with ML models by only using textual data, greatly simplifying the training and feature selection process.

We have fine-tuned BERT models with context lengths of 128 and 512, observing the model with 128-token context length performing worse. Although seBERT, a BERT-based fine-tuned model for bug classification with 128 context length, has demonstrated faster convergence; our study shows it creates a trade-off between accuracy and efficiency. This may be due to the dataset characteristics. Since our dataset has roughly 185 words per bug report it exceeds the context length of 128, while BERT-based models with 512 context size can easily accommodate entire content because a word consists approximately of two tokens [28].

B. Answers to Research Questions

RQ1: How do reasoning LLMs perform compared to prior ML and LLM classifiers with few-shot prompting?

The LLMs perform inferior to the other models in terms of binary F_1 score by significant amounts even when they are supported with in-context examples. However, since the LLMs are much larger than the BERT-based or traditional machine learning models we used, the inference time and the cost was higher. Running nine models for 2,000 samples cost around \$80, and took 15-18 hours, with the time-to-last-byte being 2-6 seconds. Very long prompts caused occasional spikes up to 26 seconds, as demonstrated in approximately 22.9 million input tokens and 3.6 million output tokens. The average cost for processing a bug report is found to be \$ 0.004. Even though the LLMs are more costly, they struggle to achieve the success of less computationally expensive models. These findings show that to increase the LLM performance different approaches should be taken in addition to few-shot prompting, and fine-tuning transformers for domain specific classification remains the most effective way for bug triage.

RQ2: What is the effect of integrating a Judge LLM into a majority-voting ensemble versus using voting alone?

The simple ensemble of SVM, Random Forest, RoBERTa delivers the strongest overall numbers (binary F_1 of 0.906)

without any LLM invocation. When those same votes are forwarded together with the five retrieved neighbors to the Judge LLM (GPT o3-mini with the judge prompt), performance drops to 0.871 F_1 (-3.5%). In other words, the Judge LLM does *not* improve the majority label but retains roughly 96% of its F_1 while adding a brief, human-readable rationale for every decision. Compared to LLM acting alone (few-shots with F_1 0.815), the judge gains 5.6% by leveraging numeric votes; compared with the majority vote, it trades a small piece of accuracy for transparency. The decrease in the classification performance of LLMs may be due to the LLMs’ tendency to classify bug reports as valid, as shown in Section IV-B. Furthermore, our analysis reveals LLM’s tendency to value the structure of the bug report and the technical information in the report more than the majority voting does.

RQ3: How does few-shot retrieval (RAG) influence LLM performance relative to zero-shot prompting?

Across all four language models the addition of five nearest-neighbor exemplars yields a consistent, double-digit improvement. For GPT o3-mini binary F_1 rises from 0.614 (zero-shot) to 0.815 (few-shot), a 20.1% gain; GPT o4-mini climbs from 0.599 to 0.759 (+16.0%); GPT-4o-mini moves from 0.531 to 0.729 (+19.8%); and even the already stronger Llama-70B improves from 0.737 to 0.797 (+6.0%). This is lower due to Llama’s already high-performance in generalized NLP tasks [44]. However, it cannot still pass gpt-o3-mini. These jumps confirm the central goal of RAG: lightweight, task-specific context can compensate for the absence of fine-tuning and substantially narrow the gap to the transformer classifiers [10], [11]. The difference between RoBERTa and few-shot o3-mini remains 9.4%. This highlights that dense retrieval alone does not close the deficit.

C. Implications for Researchers

Prompt & Retrieval Design. We fixed the retrieved neighbor count at $k = 5$, yet still observed a jump in binary F_1 when moving from zero-shot to few-shot prompting for all models. Future studies should sweep k more finely, compare dense (MiniLM) versus sparse (BM25) retrieval, and add automatic relevance filters to exclude mislabeled or off-topic neighbors.

BERT First, LLM as Explainer. Fine-tuned transformers remain the strongest stand-alone classifiers in our corpus: RoBERTa attains a binary F_1 of 0.909 with 0.948 recall, whereas the best few-shot LLM trails by almost ten points. A pragmatic deployment pattern therefore delegates the final label to a BERT-family model and reserves the LLM for post-hoc explanation and user-facing guidance. Researchers can evaluate how such “BERT-predict / LLM-explain” pipelines affect developer trust, triage latency, and reopen rates.

Learning to Weight Ensemble Evidence. The current Judge

prompt passes SVM, RF, and RoBERTa votes to the LLM as unweighted text, even though their scores differ by a non-negligible margin (RoBERTa $F_1 = 0.922$ vs. SVM $F_1 = 0.872$). Future work could provide calibrated probabilities, prompt the LLM to derive adaptive vote weights, or train a lightweight meta-learner that learns optimal weights and passes only the aggregate to the LLM, potentially closing the remaining 3–4% gap between the Judge pipeline ($F_1 = 0.871$) and the best fine-tuned transformer without sacrificing explainability, another approach may be to use LLMs as explainer.

D. Implications for Practitioners

Fine-Tuned Transformers as the Primary Filter. The RoBERTa model fine-tuned on 8,000 Firefox reports attains the highest scores in our study, making it the most reliable first-line classifier for separating valid from invalid reports. Teams can deploy it with minimal code by leveraging the HuggingFace Transformers pipelines for seamless inference in production settings [29]. This setting can be used by appending an LLM and supporting it with information about the product and the bug report.

Classical Pipelines for Low-Resource Environments. Organizations constrained to CPU-only servers can deploy a TF-IDF, SVM or RF model and still achieve $F_1 \approx 0.88$. These models train in minutes, occupy less than 50 MB on disk, and require no specialized hardware—an attractive trade-off when marginal accuracy losses are acceptable [45], [46].

Layered Explainability with Retrieval-Augmented LLMs. A retrieval-augmented GPT-o3-mini adds natural-language rationales and actionable suggestions at roughly \$0.002 per call [47]. One pragmatic workflow classifies every report with the transformer, then invokes the LLM only for invalid predictions, or transformer outputs whose confidence falls below a chosen threshold. This “classify-then-explain” pattern keeps costs predictable while providing developers and bug reporters with transparent, context-aware guidance [48].

VI. THREATS TO VALIDITY

A. Threats to Internal Validity

Stochasticity. Our results may be subject to stochastic variations from the random initialization of model weights and data splits. Each pipeline was trained and evaluated only once; multiple runs with different random seeds would be needed to establish confidence intervals for our performance metrics.

Hyperparameter Tuning. We used a standard set of hyperparameters without performing an extensive tuning process. The reported performance of any given model could potentially be improved with a more exhaustive search, which might alter the relative rankings of the classifiers.

Prompt and Generation Variability. The performance of the LLM-based reasoning is highly dependent on prompts as shown by Wei et al. [49]. We also implemented basic error handling for malformed JSON outputs [50], but inconsistent generation remains a potential source of noise in the LLM results.

B. Threats to External Validity

Single-Project Scope. Our dataset is drawn exclusively from Mozilla Firefox bug reports hosted on Bugzilla. The conventions, terminology, and types of issues reported are specific to this project and community. Consequently, our models may not generalize well to other software projects with different characteristics. This is a common limitation in bug report classification research [1], [9].

Dependency on RAG Corpus. The success of our few-shot pipeline relies on the existence of a sufficiently large and relevant corpus of historical reports for retrieval. This approach, based on retrieval-augmented generation [10], [11], would likely be less effective for new projects or niche domains with sparse historical data, limiting its general applicability.

Model Availability. Our LLM experiments were constrained to models available on the Azure AI Studio platform [34]. The results may not be representative of other powerful LLMs, such as newer models from Google [35] or Anthropic [36], which might exhibit different performance characteristics on this task.

C. Threats to Construct Validity

Temporal Drift. The collected data was between January 1, 2020, and April 13, 2020. Software systems and their reporting practices evolve over time. Our models, trained on this static snapshot, may lose accuracy when applied to future bug reports due to this “concept drift”, a known challenge in systems that rely on historical software data [20].

Lack of Formal Explainability Evaluation. While a key contribution is the generation of explanations by the Judge LLM, we did not formally evaluate their quality or utility to developers. A rigorous, human-centric study would be required to validate whether these explanations are genuinely useful and trustworthy, following principles outlined in work on interpretable AI and human-AI interaction [42], [48].

VII. CONCLUSION AND FUTURE WORK

We introduced a retrieval-augmented, multi-stage pipeline that unifies three model families for Bugzilla triage: classical ML models, BERT-based classifiers, and large-language models. We found that BERT-based models achieve the best performance even though they require less computation than LLMs. Furthermore, we have found that LLMs perform poorly without additional information and prompting techniques. Even though LLM performance significantly improves with these techniques, they still tend to err on the side of classifying bug reports as valid.

In the future, we plan to investigate various information retrieval pipelines for LLMs, and to verify LLM explanations to further tighten the gap between explainability and accuracy. Furthermore, to improve the bug tracking process for invalid bug reports, we plan to extend this paper to subclassify invalid bug reports and provide informed suggestions for users. Finally, we aim to deploy an architecture in which fine-tuned BERT models act as the primary classifiers and an LLM provides explanations for predictions.

REFERENCES

- [1] Y. Fan, X. Xia, D. Lo, and A. E. Hassan, "Chaff from the wheat: Characterizing and determining valid bug reports," *IEEE Transactions on Software Engineering*, vol. 46, no. 5, pp. 495–525, 2020. DOI: 10.1109/TSE.2018.2864217.
- [2] C. Eren, K. Şahin, and E. Tüzün, "Analyzing bug life cycles to derive practical insights," in *Proceedings of the 27th International Conference on Evaluation and Assessment in Software Engineering*, ser. EASE '23, Oulu, Finland: Association for Computing Machinery, 2023, pp. 162–171, ISBN: 9798400700446. DOI: 10.1145/3593434.3593504. [Online]. Available: <https://doi.org/10.1145/3593434.3593504>.
- [3] G. Antoniol, K. Ayari, M. Di Penta, F. Khomh, and Y.-G. Gueheneuc, "Is it a bug or an enhancement? a text-based approach to classify change requests," in *Proc. CASCON 2008*, 2008, 23:1–23:15. DOI: 10.1145/1463788.1463819.
- [4] M. S. Zanetti, I. Scholtes, C. J. Tessone, and F. Schweitzer, "Categorizing bugs with social networks: A case study on four open source software communities," in *2013 35th International Conference on Software Engineering (ICSE)*, 2013, pp. 1032–1041. DOI: 10.1109/ICSE.2013.6606653.
- [5] A. F. Otoom, S. Al-jdaeh, and M. Hammad, "Automated classification of software bug reports," in *Proc. 9th Int. Conf. on Information Communication and Management (ICICM '19)*, 2019, pp. 17–21. DOI: 10.1145/3357419.3357424.
- [6] N. Pandey, D. K. Sanyal, A. Hudait, and A. Sen, "Automated classification of software issue reports using machine learning techniques: An empirical study," *Innovative Systems and Software Engineering*, vol. 13, no. 4, pp. 279–297, 2017. DOI: 10.1007/s11334-017-0294-1.
- [7] Q. Meng and J. Visser, "Which bug reports are valid and why? using the bert transformer to classify bug reports and explain their validity," in *Proc. 4th European Symposium on Software Engineering (ESSE '23)*, 2024, pp. 52–60. DOI: 10.1145/3651640.3651648.
- [8] G. Aracena, K. Luster, F. Santos, I. Steinmacher, and M. A. Gerosa, "Applying large language models to issue classification," in *Proc. ACM/IEEE Int. Workshop on NL-Based Software Engineering (NLBSE)*, 2024, pp. 57–60. DOI: 10.1145/3643787.3648043.
- [9] M. Laiq, N. b. Ali, J. Börstler, and E. Engström, "A comparative analysis of ml techniques for bug report classification," *J. Syst. Softw.*, vol. 227, no. C, Jun. 2025, ISSN: 0164-1212. DOI: 10.1016/j.jss.2025.112457. [Online]. Available: <https://doi.org/10.1016/j.jss.2025.112457>.
- [10] P. Lewis et al. "Retrieval-augmented generation for knowledge-intensive nlp tasks," in *Proc. NeurIPS 2020*, 2020, pp. 9459–9474. DOI: 10.48550/arXiv.2005.11401. [Online]. Available: <https://doi.org/10.48550/arXiv.2005.11401>.
- [11] K. Guu, K. Lee, Z. Tung, P. Pasupat, and M.-W. Chang, "Realm: Retrieval-augmented language model pre-training," in *Proc. ICML 2020*, 2020, pp. 3929–3938. DOI: 10.48550/arXiv.2002.08909.
- [12] W. Wang, F. Wei, L. Dong, H. Bao, N. Yang, and M. Zhou, "Minilm: Deep self-attention distillation for task-agnostic compression of pre-trained transformers," in *Proc. NeurIPS 2020*, 2020, pp. 5776–5788. DOI: 10.48550/arXiv.2002.10957.
- [13] J. Johnson, M. Douze, and H. Jégou, *Billion-scale similarity search with gpus*, 2017. arXiv: 1702.08734 [cs.CV]. [Online]. Available: <https://arxiv.org/abs/1702.08734>.
- [14] C. Sun, D. Lo, X. Wang, J. Jiang, and S.-C. Khoo, "A discriminative model approach for accurate duplicate bug report retrieval," in *2010 ACM/IEEE 32nd International Conference on Software Engineering*, vol. 1, 2010, pp. 45–54. DOI: 10.1145/1806799.1806811.
- [15] B. Kucuk, I. Hanhan, and E. Tuzun, "Characterizing duplicate bugs: Perceptions of practitioners and an empirical analysis," *J. Softw. Evol. Process*, vol. 35, no. 11, Nov. 2023, ISSN: 2047-7473. DOI: 10.1002/smr.2446. [Online]. Available: <https://doi.org/10.1002/smr.2446>.
- [16] J. von der Mosel, A. Trautsch, and S. Herbold, "On the validity of pre-trained transformers for natural language processing in the software engineering domain," *IEEE Transactions on Software Engineering*, vol. 49, no. 4, pp. 1487–1507, 2023. DOI: 10.1109/TSE.2022.3178469.
- [17] W. Wang, Y. Wang, S. Joty, and S. C. Hoi, "Rap-gen: Retrieval-augmented patch generation with codet5 for automatic program repair," in *Proc. ESEC/FSE 2023*, 2023, pp. 146–158. DOI: 10.1145/3611643.3616256.
- [18] A. Arzanipour, R. Behnia, R. Ebrahimi, and K. Dutta, "Rag security and privacy: Formalizing the threat model and attack surface," *arXiv preprint arXiv:2509.20324*, 2025. DOI: 10.48550/arXiv.2509.20324.
- [19] M. Foundation, *Bugzilla: Official documentation*, <https://bugzilla.readthedocs.io/en/latest/>, Accessed: 2025-07-13; contains official definitions and workflow guidelines for Bugzilla bugs, 2025.
- [20] T. Zimmermann, N. Nagappan, P. J. Guo, and B. Murphy, "Characterizing and predicting which bugs get re-opened," in *Proc. 34th Int. Conf. Software Engineering (ICSE)*, 2012, pp. 1074–1083. DOI: 10.1109/ICSE.2012.6227112.
- [21] A. Adhikari, A. Ram, R. Tang, and J. Lin, *Docbert: Bert for document classification*, 2019. arXiv: 1904.08398 [cs.CL]. [Online]. Available: <https://arxiv.org/abs/1904.08398>.
- [22] R. Kohavi, "A study of cross-validation and bootstrap for accuracy estimation and model selection," in *Proc. 14th Int. Joint Conf. Artificial Intelligence (IJCAI'95)*, Accessed: 2025-10-26, 1995, pp. 1137–1143. [Online].

- Available: <https://dl.acm.org/doi/abs/10.5555/1643031.1643047>.
- [23] M. Buda, A. Maki, and M. A. Mazurowski, "A systematic study of the class imbalance problem in convolutional neural networks," *Neural Networks*, vol. 106, pp. 249–259, 2018. DOI: 10.1016/j.neunet.2018.07.011.
 - [24] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "Bert: Pre-training of deep bidirectional transformers for language understanding," in *Proc. NAACL-HLT 2019*, 2019, pp. 4171–4186. DOI: 10.18653/v1/N19-1423.
 - [25] Y. Liu et al. *Roberta: A robustly optimized bert pre-training approach*, 2019. arXiv: 1907.11692 [cs.CL]. [Online]. Available: <https://arxiv.org/abs/1907.11692>.
 - [26] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, *Distilbert, a distilled version of bert: Smaller, faster, cheaper and lighter*, 2020. arXiv: 1910.01108 [cs.CL]. [Online]. Available: <https://arxiv.org/abs/1910.01108>.
 - [27] Z. Feng et al. *Codebert: A pre-trained model for programming and natural languages*, 2020. arXiv: 2002.08155 [cs.CL]. [Online]. Available: <https://arxiv.org/abs/2002.08155>.
 - [28] P. Rust, J. Pfeiffer, I. Vulić, S. Ruder, and I. Gurevych, "How good is your tokenizer? on the monolingual performance of multilingual language models," in *Proc. ACL-IJCNLP 2021*, 2021, pp. 3118–3135. DOI: 10.18653/v1/2021.acl-long.243.
 - [29] T. Wolf et al. "Transformers: State-of-the-art natural language processing," in *Proc. EMNLP 2020 (System Demonstrations)*, 2020, pp. 38–45. DOI: 10.18653/v1/2020.emnlp-demos.6.
 - [30] S. Yao et al. "React: Synergizing reasoning and acting in language models," *ArXiv*, vol. abs/2210.03629, 2022. [Online]. Available: <https://api.semanticscholar.org/CorpusID:252762395>.
 - [31] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, "Language models are unsupervised multitask learners," *OpenAI Blog*, 2019. [Online]. Available: https://cdn.openai.com/better-language-models/language_models_are_unsupervised_multitask_learners.pdf.
 - [32] T. B. Brown et al. "Language models are few-shot learners," in *Proc. NeurIPS 2020*, 2020, pp. 1877–1901. DOI: 10.48550/arXiv.2005.14165.
 - [33] C. D. Manning, P. Raghavan, and H. Schütze, *Introduction to Information Retrieval*. Cambridge University Press, 2008, ISBN: 978-0-521-86571-5. [Online]. Available: <https://www.cambridge.org/core/books/introduction-to-information-retrieval/669D108D20F556C5C30957D63B5AB65C>.
 - [34] Microsoft Corporation, *Azure AI Studio*, <https://azure.microsoft.com/en-us/products/ai-studio/>, Accessed: 2025-07-22, 2025.
 - [35] Google DeepMind, *Gemini 2.5 pro*, <https://deepmind.google/models/gemini/pro/>, Accessed: 2025-10-15, 2025.
 - [36] Anthropic, *Introducing claude 4*, <https://www.anthropic.com/news/claude-4>, Accessed: 2025-10-15, May 2025.
 - [37] OpenAI, *Models*, <https://platform.openai.com/docs/models>, Accessed: 2025-10-15, 2025.
 - [38] Meta, *Introducing meta llama 3: The most capable openly available llm to date*, <https://ai.meta.com/blog/meta-llama-3/>, Accessed: 2025-10-15, Apr. 2024.
 - [39] OpenAI, *Introducing openai o3 and o4-mini*, <https://openai.com/index/introducing-o3-and-o4-mini/>, Accessed: 2025-10-15, Apr. 2025.
 - [40] OpenAI, *GPT-4o mini: Advancing cost-efficient intelligence*, <https://openai.com/index/gpt-4o-mini-advancing-cost-efficient-intelligence/>, Accessed: 2025-10-15, Jul. 2024.
 - [41] G. Brown, J. Wyatt, R. Harris, and X. Yao, "Diversity creation methods: A survey and categorisation," *Information Fusion*, vol. 6, no. 1, pp. 5–20, 2005. DOI: 10.1016/j.inffus.2004.04.004.
 - [42] F. Doshi-Velez and B. Kim, "Towards a rigorous science of interpretable machine learning," *arXiv: Machine Learning*, 2017. [Online]. Available: <https://api.semanticscholar.org/CorpusID:11319376>.
 - [43] D. Larionov et al. *Deepseek-r1 vs. o3-mini: How well can reasoning llms evaluate mt and summarization?* 2025. arXiv: 2504.08120 [cs.CL]. [Online]. Available: <https://arxiv.org/abs/2504.08120>.
 - [44] A. Grattafiori et al. "The Llama 3 herd of models," *arXiv preprint arXiv:2407.21783*, 2024. DOI: 10.48550/arXiv.2407.21783. [Online]. Available: <https://arxiv.org/abs/2407.21783>.
 - [45] T. Joachims, "Text categorization with support vector machines: Learning with many relevant features," in *Proc. ECML 1998*, 1998, pp. 137–142. DOI: 10.1007/BFb0026683.
 - [46] F. Pedregosa et al. *Scikit-learn: Machine learning in python*, 2018. arXiv: 1201.0490 [cs.LG]. [Online]. Available: <https://arxiv.org/abs/1201.0490>.
 - [47] Microsoft, *Azure openai service pricing*, <https://learn.microsoft.com/azure/cognitive-services/openai/pricing>, Accessed: 2025-07-20, 2024.
 - [48] S. Amershi et al. "Guidelines for human-ai interaction," in *Proc. CHI 2019*, 2019, pp. 1–13. DOI: 10.1145/3290605.3300233.
 - [49] J. Wei et al. "Chain-of-thought prompting elicits reasoning in large language models," in *Proc. NeurIPS 2022*, 2022, 1800:1–1800:14. DOI: 10.48550/arXiv.2201.11903.
 - [50] OpenAI, *Introducing structured outputs in the api*, Accessed: 2025-10-26, 2024. [Online]. Available: <https://openai.com/blog/introducing-structured-outputs-in-the-api>.